Microservices:

- an architectural style that splits an application into small, independent services, each handling a specific business function.
- These services run in their own processes, often with their own databases, and communicate via APIs (typically REST or messaging).
- Unlike a monolithic architecture, where all code is one deployable unit, microservices allow each service to be developed, deployed, and scaled separately, enabling changes to individual services (e.g., updating the payment service) without affecting the entire system (e.g., the user service).

Aspect	Monolithic Architecture	Microservices Architecture
Code Structure	Single, tightly coupled code base	Modular and decoupled components
Deployment	Entire system deployed as one unit	Each service deployable separately
Updates	Risky; changes affect the entire system	Safe; changes impact only specific services
Flexibility	Limited due to interwoven components	High due to independent service management
Fault Isolation	A failure in one component can affect the entire system	Issues remain isolated to specific services. failure in one service usually won't crash the entire system
Scaling	Inflexible; scaling requires updating the whole system	Flexible; individual services can scale independently
		Agility: teams can develop, deploy and redeploy services without affecting others
		Enable small teams to own individual services (aligning with Agile and DevOps practices).
		Let organizations iterate faster and adopt new technologies per service

Core principles of microservices architecture:

- **Single Responsibility:** Each service focuses on one business capability.
- **Bounded Contexts:** Services align with clear domain boundaries.
- **Independent Deployability:** Services can be deployed without affecting others.
- **Decentralized Governance:** Teams choose the best technology for each service.
- **Failure Isolation:** Failure in one service should not bring down the entire system.
- **Scalability:** Services can be scaled independently.
- **Consumer-First Design:** APIs should be designed with consumers in mind.

Anti-patterns in microservices design:

- Services shared a single database,
- Services are too large and violate the single responsibility principle.
- Services are broken down too finely, leading to increased complexity.
- Require excessive inter-service calls to complete a single tass,
- Long sequences of synchronous calls between services, risking cascading failures.
- Deploying multiple services as one-unit, negating independence.

Inter-service communication (microservices communicate with each other):

Synchronous communication REST APIs (HTTP), gRPC (HTTP/2, Protocol Buffers).

- Client makes a request and waits for a response but introduces tight coupling, latency and potential for cascading failures.
- Choose when an immediate response is needed, and the calling service is dependent on the result of the called service (e.g., getting user profile data, processing a payment synchronously).

Asynchronous messaging: Message Queues or message broker (RabbitMQ, Kafka, ActiveMQ), Event Buses.

- Services communicate via messages or events. One service publishes an event or message, and others consume it when ready. Provides better decoupling, resilience, and scalability.
- Choose when services don't need an immediate response, can process requests independently, or need to communicate with multiple services without waiting for all responses (e.g., sending email notifications, updating inventory after an order, event-driven workflows).

For example, placing an order might synchronously call the payment service, while stock updates could be done via asynchronous events. Both methods are common: REST/HTTP for request/response APIs and a message bus for events or tasks.

1. **What is event-driven architecture, and how does it complement microservices?**

   In an **Event-driven architecture**, services communicate by emitting and reacting to events rather than direct API calls. This promotes: DSRRF

   - **Decoupling:** Services only know events they produce or consume, not each other.
   - **Scalability:** Event consumers can scale independently.
   - **Resilience**: Services process events asynchronously, if a consumer is down, events are processed later when it's available.
   - **Real-time Processing:** Reacts to changes instantly.
   - **Flexibility:** Easy to add new event producers or consumers.

2. **What are the Key Challenges of Adopting and Operating Microservices? OIDT**

   - **Operational Overhead:** More services increase complexity in monitoring, logging, and deployments compared to monoliths.
   - **Inter-Service Communication:** APIs or messaging introduce network latency, failures, and need for reliable patterns like retries or circuit breakers**.**
   - **Data Consistency:** Separate database for each service complicates consistency, requiring eventual consistency, sagas, or event-driven approaches.
   - **Testing Complexity:** Distributed testing (integration, contract, end-to-end) is harder than monoliths.
   - **Migration Challenges:**
     - **Data Migration:** Moving from a monolithic to per-service databases require complex refactoring and synchronization.
     - **Incremental Transition:** Gradually breaking a monolith into microservices requires maintaining both systems simultaneously, increasing operational overhead.
     - **Organizational Restructuring:** Teams must align with service boundaries, often disrupting structure (Conway's Law).

3. **What is an API gateway, and why is it important in a microservices ecosystem? why is it used in a microservices architecture?**

   An API gateway:

   - Acts as a single-entry point, routing client requests to the appropriate microservice.
   - Simplifies client interaction with one URL and centralizes security, logging, and monitoring.
   - Handles: RARARIT
     - Request routing
     - Authentication/authorization
     - Response aggregation (combining responses from multiple services).
     - Avoid exposing each service directly to clients
     - Reduces the complexity for clients and centralizes common tasks.
     - Implement rate limiting, logging, and monitoring.
     - Translate protocols (e.g., from REST to gRPC).

   For example, tools like Spring Cloud Gateway making the architecture more secure and manageable.

4. **Explain the concept of service discovery and how to implement service discovery in a microservices environment?**

   Service discovery

   - enables microservices to automatically register and locate each other ((e.g., through tools such as Netflix Eureka or Spring Cloud Service Discovery) without hard-coded endpoints, supporting scalability and resilience.
   - enables clients or services to find IP address and port of service instances, accommodating auto-scaling as instances register or unregister automatically.

   Implementation Approaches:

   - **Client-Side Discovery:** Services query a registry (e.g., Eureka, Consul) to retrieve a list of available instances and route requests directly to them. This approach gives clients control over load balancing but requires integration with the registry.
   - **Server-Side Discovery:** A load balancer or API gateway (e.g., Kubernetes Service, Envoy) handles requests, dynamically routing traffic to available services. This simplifies client logic and centralizes traffic management.

5. **Describe the "Database per Service" pattern and its trade-offs.**
Each microservice has and can access only its own private database and expose data via their APIs.
   - **Advantages: sd**
     - services choose their own database technology.
     - database failure affects only one service which promotes resilience.
   - **Disadvantages:** dhd
     - data consistency challenges (needs Sagas or eventual consistency for data spanning multiple services),
     - higher database management overhead,
     - difficulty with cross-service queries or joins (requires API composition or data duplication).

6. **How do you maintain data consistency across microservices?**
Data consistency is typically managed through eventual consistency models. ***Eventual consistency*** means data may be temporarily out of sync but will align over time through asynchronous updates. It prioritizes availability and performance by allowing services to operate independently, updating asynchronously and resolving inconsistencies as the system stabilizes.
Techniques include:
   - **Asynchronous Communication:** Services publish change events instead of waiting on direct calls. Each service consumes and updates its own data on its own schedule, so updates propagate through the system over time and all services eventually converge on the same data.
   - **Event-Driven Architecture:** Services respond to events (like a data update) from other services. This decouples them, letting each service update its own data on its own schedule without requiring real-time coordination.
   - **Message Queues:** If a service is temporarily unavailable, the queue holds the message until the service is back online, preventing data loss and supporting consistency over time.
   - **Sagas:** Sagas break the process into smaller, local transactions. If one step fails, sagas trigger reversing earlier steps to keep the system consistent.

7. **How do microservices improve high availability, resilience and handle faults?**
Microservices resilience is often achieved with patterns like: rtbcs

i. retries,	ii. timeouts	iii. bulkheads

iv. circuit breakers (The circuit breaker pattern)
   - fault-tolerance mechanism that prevents repeated failed requests to an unresponsive service by temporarily blocks (short-circuits) calls to the failing service, allowing time for recovery before retrying.
   - Libraries like Hystrix, Resilience4j, and built-in Kubernetes mechanisms help implement this pattern effectively.

Importance in Microservices:
- Prevents cascading failures – Stops excessive retries from overwhelming dependent services.
- Enhances resilience – Improves system stability by handling faults gracefully.
- Optimizes performance – Redirects traffic or returns fallback responses instead of waiting for timeouts.

v. Saga Pattern (What is the Saga pattern for distributed transactions)?
   - manages distributed transactions without deploying all services together or using two-phase commits.
   - manage distributed transactions by breaking them into local steps, each updating a service and emitting events. If a step fails, compensating actions rollback previous steps.
   - follow choreography (event-based) or orchestration (central coordinator) to maintain consistency.
   Example: If payment succeeds but inventory fails, a refund is issue

Two Main Approaches

1. ***Choreography:*** ○ *No central coordinator.Each service listens for events and reacts accordingly.* ○ *Simpler but harder to track and debug.*	2. ***Orchestration:*** ○ *A central orchestrator tells each service what to do.* ○ *Easier to manage and monitor, but introduces a single point of control.*

Example: Order Processing Saga
Let's say you're building an online store. The flow might look like this:
1. ***Order Service****: Receives the order → saves it → emits OrderCreated event.*
2. ***Payment Service****: Listens for OrderCreated → charges the customer → emits PaymentCompleted.*
3. ***Inventory Service****: Listens for PaymentCompleted → reserves items → emits InventoryReserved.*

4. ***Shipping Service**: Listens for InventoryReserved → schedules delivery.*

*If any step fails, the previous services trigger **compensating actions**:*

- *Payment failed? → Cancel order.*
- *Inventory not available? → Refund payment.*

8. **How do you handle data duplication or denormalization across services?**

Data duplication is sometimes necessary to avoid complex cross-service joins or synchronous calls. For example, a "Product" service might duplicate some product information (like name and price) in an "Order" service's database. When the source data changes (e.g., product price), the change is propagated asynchronously (e.g., via an event) to update the duplicated data in other services. This requires careful management to maintain eventual consistency.

9. ***When defining microservice boundaries, we should rely on three key principles: (BTT)***

- **Business capabilities**: I use Domain-Driven Design to identify bounded contexts—distinct areas of the business domain.
  - 'Order Management' is a bounded context and becomes its own microservice.

    *A **bounded context** defines a clear boundary within a specific business domain, encapsulating its own data model and logic. This isolation reduces dependencies, clarifies service responsibilities, and enables easier development and scaling of microservices.*
- **Team Alignment**: Following Conway's Law, align services to organizational structure. Teams own and manage the services that reflect their domain.
  - 'Order Management' service is owned by the orders team to minimize coordination.
- **Transactional boundaries**: Keep related logic and data within a single service to avoid distributed transactions. This ensures autonomy and easier scaling.
  - 'Order Management' handles all order-related processing.

10. **What are the considerations for logging and monitoring in a microservices system?**

Monitoring involves:

- **Distributed Tracing**: Zipkin/Jaeger - tracking request flows and identify performance issues.
- **Centralized Logging:** ELK/Splunk - aggregate logs in systems for analysis.
- **Metrics Collection:** Prometheus/Grafana - monitor CPU, memory, latency, and errors.
- **Log Correlation:** Use trace IDs to debug requests across services.
- **Health Checks**: Expose health endpoints for orchestration tools (e.g., Kubernetes) to monitor service status.

Cross-cutting concerns in microservices are managed using sidecar services or tools like (Zipkin, ELK, Prometheus) which enable unified or centralized monitoring without mixing concerns with business logic.

11. **What is the sidecar pattern, and how is it used in microservices?**

The sidecar pattern involves deploying a helper (auxiliary) service alongside the primary (main) service in the same container or pod. This helper service handles logging, security, or monitoring—offloading common concerns from the primary service and simplifying its design.

12. **What is the role of Observability in a microservices architecture?**

Observability is the ability to understand the internal state of a system by examining external outputs (traces , logs, metrics). Observability is vital for: (DIUM)

- ➢ Debugging issues across service boundaries.
- ➢ Identifying bottlenecks.
- ➢ Understanding system behaviours in production.
- ➢ Monitoring and alerting.

13. **What are some common security considerations in microservices?**

- **API Gateway Security:** Authenticating and authorizing external requests at the gateway.
- **Inter-service Security:** Securing communication *between* services (e.g., mTLS, JWT propagation).

How do you handle authentication and authorization in a microservices system?

i. **Authentication:** Often handled at the API Gateway or a dedicated Authentication Service. The user logs in once and gets a token (like JWT). This token proves their identity when making requests.

ii. **Authorization:**
   - **Gateway/Edge Enforcement:** The API Gateway does a basic authorization check using the token.
   - **Token Propagation:** Passing the authentication token (e.g., JWT) to downstream services.
   - **Internal Authorization (Service-Level Checks):** Each microservice verifies the token and decides what the user can access. Sometimes, services call an **Authorization Service** for deeper checks.

14. How do you approach testing in a CI/CD pipeline for microservices?

A multi-layered testing strategy is essential:

i. **Unit Tests:** Testing individual components/classes within the service.

ii. **Component Tests:** Testing a single service in isolation but with its real dependencies (database, etc.).

iii. **Integration Tests:** Testing interactions between components within a service or between a service and its database/external dependencies (often with mocks/stubs).

iv. **Contract Tests -** verifies, without deploying all services together, that the agreement (contract) between a service provider and consumer—such as expected API requests and responses—is upheld. It ensures correct interactions and catches integration issues early by testing in isolation.

v. **End-to-End (E2E) Tests:** Testing user flows/journey across multiple services (use sparingly as they can be fragile).

15. How do you handle versioning of microservices APIs? Common strategies: (UHCV)

- **URL Versioning:** Version number in the URL (e.g., /api/v1/users). Simple but violates REST principles (URI should identify a resource, not its representation).
- **Header Versioning**: Version specified in a *custom HTTP header*.
- **Content Negotiation**: Version via the *Accept header*. Considered a more RESTful approach.
- **Versioning via API Gateway:** Versioning is handled by gateway, which routes requests to the correct service version.

16. How do you handle configuration management for microservices?

- Configurations (database credentials, external service URLs, feature flags) should be externalized and managed centrally. Services fetch configs from the central store at startup or dynamically.
- Use Spring Cloud Config, Consul, etcd, or Kubernetes ConfigMaps/Secrets to store configs.

17. How do you manage dependencies between microservices during development and deployment?

- **Development:** UUM
  - Use clear API contracts.
  - Use integration test environments to test interactions between services.
  - Mock or stub dependencies during local development and unit testing.
- **Deployment:** DUUO
  - Use backward compatibility in APIs or versioning to handle situations where services are updated at different times.
  - Use contract testing to catch integration issues before deployment.
  - Deploy services independently.
  - Orchestration platforms (like Kubernetes) help manage service dependencies and rollout strategies.

18. What role does orchestration play in a microservices-based ecosystem?

Orchestration tools (like Kubernetes) manage the deployment, scaling, and networking of containerized microservices. They automate many operational tasks, reducing complexity and allowing teams to focus more on developing business logic.

19. What considerations should be taken for data storage in a microservices architecture?

- Each microservice owns its data store/database to ensure loose coupling.
- Embrace eventual consistency rather than strong consistency across services. Use sagas or event-driven architecture.
- Use messaging systems (like Kafka or RabbitMQ) to propagate data changes asynchronously. Replicate data only where necessary to reduce complexity.

~~**20. What is the importance of automation in a microservices environme nt?**~~

~~i. Automation is critical due to the increased number of services and deployments. This includes:~~

ii. ~~**Automated Builds:** Compiling code and creating artifacts.~~
iii. ~~**Automated Testing:** Running unit, integration, and contract tests.~~
iv. ~~**Automated Deployments:** Using CI/CD pipelines to deploy services to various environments.~~
v. ~~**Automated Infrastructure Provisioning:** Using Infrastructure as Code (IaC) tools (like Terraform, CloudFormation) to manage environments.~~
vi. ~~**Automated Monitoring and Alerting:** Detecting issues and notifying teams. Automation reduces manual effort, increases speed, reduces errors, and enables frequent, reliable deployments.~~

~~**21. What are the challenges of testing microservices compared to a monolith?**~~

- ~~**Integration Testing:** Testing interactions between multiple services is more complex and requires managing dependencies and environments.~~
- ~~**Contract Testing:** Ensuring that services adhere to their API contracts with consumers helps prevent integration issues between independently deployable services.~~
- ~~**End-to-End Testing:** Testing a user journey across many services can be fragile and slow.~~
- ~~**Managing Test Environments:** Setting up and tearing down environments with multiple services can be complex.~~

~~**22. Can you describe a real-world scenario where you implemented microservices to overcome scaling or maintenance challenges?**~~

DOCKER AND DOCKER SWARM

1. **What is Docker and what are containers? How do containers differ from virtual machines?**

Docker is a containerization platform that packages applications with all their dependencies into containers. This enables consistent deployment across environments (development, testing, and production environments) eliminating the "works on my machine" issue. This ultimately improving scalability, reproducibility, and portability.

Containerization isolates an application and its dependencies from the host system by encapsulating them in a container. This ensures that the application runs uniformly regardless of differences in operating systems or underlying infrastructure.

- ***container*** is an isolated, lightweight environment running on the host's operating system kernel. Unlike a ***VM*** (which includes a full guest OS), containers share the host OS kernel but maintain their own user-space.
- ***containers*** are much more lightweight and start faster than ***VMs***.
- ***containers*** provide portability and isolation without the overhead of a full ***VM*** OS.

Main Advantages: (CIPERS)

- **Consistency:** Containers ensures applications run the same way in any environment (development, testing, prod).
- **Isolation:** Containers are isolated from each other and the host system, preventing conflicts.
- **Portability:** Containers can be easily moved between different environments.
- **Efficiency:** Containers are much more lightweight and start faster than VMs
- **Resource Efficiency:** Container share the host OS kernel.
- **Simplified Deployment:** Standardized packaging and deployment process.

1. **What is a Docker image and how does it differ from a container?**

A Docker image. bcb

- blueprint or read-only template used to create containers.
- contains the application code, configuration, libraries and dependencies,.
- built from a Dockerfile and are composed of layers.

A container: rly

- running instance of an image.
- live environment where the application executes.
- You can start, stop, run, and delete containers

In other words:

- ***images*** are immutable and stored; ***containers*** are live, mutable runtime environments. You can create many containers from the same image; each is isolated with its own writable layer. When you run docker run, Docker takes the image, adds a top writable layer, and executes it as a container

2. **What is a Dockerfile and how do you use it?**

A Dockerfile is: asrm

- a text file that contains instructions for building a Docker image.
- specifies the base image, dependencies, and commands to configure the application.
- *read by Docker to automatically build an image layer by layer. Each instruction creates a new layer in the image*.
- make image builds repeatable and version-controlled.

Best practices for writing a Dockerfile: (How do you optimize a Docker image?)

a) Choosing a minimal base image.
b) Cleaning up temporary files.
c) Combining commands to reduce layers.
d) Using multi-stage builds to separate build and runtime environments.
e) Specifying correct entry points and commands.

FROM openjdk:21-jdk **COPY . /app** **WORKDIR /app** **RUN ./mvnw package** **CMD ["java", "-jar", "target/myapp.jar"]**	Essential instructions include: • FROM: Specifies the base image the build starts from. • WORKDIR: Sets the working directory for subsequent COPY, RUN, ENTRYPOINT, CMD and ADD instructions. • COPY: Copies files or directories from the source path on the host to the destination path in the image. • RUN: Executes commands during the image build process. • EXPOSE: Informs Docker that the container listens on the specified network ports at runtime. • ENTRYPOINT: Configures a container to run as an executable. Often used in conjunction with CMD for specifying default parameters. • CMD: Provides default arguments for executing a container. Can be overridden when running the container. • ADD: Similar to COPY but can also extract tar files from a URL or source.

~~A **.dockerignore** file excludes specified files and directories from the build context, improving build speed, reducing image size, and preventing sensitive data from being copied into the image. Just like .gitignore in Git.~~

~~**3. Explain Docker's layered architecture.**~~

~~Docker images are built in layers; each layer represents an instruction or change in the image. These layers make images lightweight and efficient since identical layers can be shared and cached across multiple images, reducing storage use and speeding up deployments.~~

4. **How do you create a Docker image for a Java Spring Boot application?**
   a. Create a Dockerfile in the root of the Spring Boot project and approach it using a multi-stage build:

      Stage 1 (Builder): Use a base image with Java and Maven/Gradle to build JAR/WAR.

```
FROM docker.io/library/openjdk:21-jdk-slim AS builder
WORKDIR /app
COPY . .
RUN ./gradlew clean build --no-daemon
```

      Stage 2 (Runtime): Use a lightweight JRE base image. Copy the build artifact from the builder stage into this runtime image. Set the ENTRYPOINT or CMD to run the JAR/WAR.

```
FROM eclipse-temurin:21-jre-jammy
WORKDIR /app
COPY --from=builder /app/build/libs/email-0.0.1-SNAPSHOT.jar ./app.jar

ENV SPRING_PROFILES_ACTIVE=prod
EXPOSE 8888
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
// CMD ["java", "-jar", "/app/your-app.jar"].`
```

5. **How would you create a Docker image for an Angular application?**
   Stages are:

      Stage 1 (Builder): Use a Node.js base image. Copy the Angular source code, install dependencies (npm install), and build the application (ng build --configuration=production).

```
FROM node:18 AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
RUN ng build --configuration=production
```

      Stage 2 (Web Server): Use a lightweight web server image (like Nginx or Apache). Copy the built Angular application files (/dist folder) from the builder stage into the web server's content directory. Configure the web server to serve the static files.

```
FROM nginx:alpine
COPY --from=builder /app/dist/your-app /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

6. **How do you expose ports for a Docker container?**
   you can use the -p (or --publish) flag with the docker run command. This maps a port on your host machine to a port inside the container.

      Syntax: **docker run -p <host_port>:<container_port> <image_name>**
      Example: **docker run -p 8080:80 nginx**
      This maps port 80 inside the container (the default port Nginx listens on) to port 8080 on the host, so you can access the web server by visiting http://localhost:8080.

7. **Name some essential Common Docker CLI commands include:**
   a) docker run (create and start a container from an image)
   b) docker stop <container> (stop a running container),
   c) docker images ls (list images),
   d) docker contains ls [docker ps (list running containers)] and docker ps -a (all containers)
   e) docker rmi <image> (remove an image)
   f) docker rm <container> (remove a container)
   g) docker build -t name . (build an image from a Dockerfile)s
   h) docker logs <container> (view a container's logs).
   i) docker exec -it <container> sh (open a shell in a running container)
   These are used daily to manage Docker images and containers. (No external reference needed; standard usage.

8. **How do you handle stateful applications (like databases) with Docker?**
   For stateful applications, you use **Docker Volumes** to persist the data outside the container. The container writes data to a mounted volume, so if the container is removed or replaced, the data remains on the volume and can be attached to a new container instance.
   In orchestration platforms like Kubernetes, uses concepts like Persistent Volumes and StatefulSets for managing stateful applications.

9. **What is Docker Compose and how is it useful?** Docker Compose:
   - is a tool (and uses a YAML configuration) for defining and running multi-container Docker applications.
   - enables you to manage multiple containers with their dependencies networks and volumes in one declarative file.
   - You define your application's services, networks, and volumes in a docker-compose.yml file.
   - You can then use a single command (docker compose up) to build, create, and start all the services defined in the file.
     Ideal for: (NDE)
     - Centralized Configuration – No need for multiple docker run commands.
     - Service Connectivity – Define how containers interact (e.g., web & database).
     - Scaling Made Easy – Easily deploy multiple instances.
     - ~~Simplified Local & CI Management – Streamlines workflows.~~

10. **Docker Network Types: BHNOM**
    Docker Networking allows containers to communicate within a single host or across multiple hosts, with overlay networks (especially in Swarm mode) facilitating communication in a clustered environment. Types are:
    - **Bridge (default)** – Isolated network for containers on the same host. Ideal for multi-container setups on a single machine.
    - **Host** – Shares the host's network stack. Used for performance-sensitive apps needing direct host access.
    - **None** – No network connectivity. Suitable for isolated containers without external communication.
    - **Overlay** – Enables multi-host communication in Docker Swarm, creating a distributed network.
    - **Macvlan** – Assigns a MAC address, making the container appear as a physical device. Useful for legacy apps or direct network presence.

11. **What role do overlay networks play in Docker Swarm?**
    Overlay networks enable containers running on different Docker hosts to communicate securely. Swarm mode, overlay networks help in isolating network segments and facilitating secure, cross-host container communication without manual network configuration.

12. **What is Docker Swarm and what is its purpose?**
    Docker Swarm
    - is Docker's native clustering and orchestration solution.
    - allows multiple Docker hosts to join and manage containers as a single virtual system.
    - used to orchestrate containers at scale, allowing you to deploy, scale, and manage a collection of containers as a "service" across multiple machines.

    While Kubernetes offers extensive features and is more complex, Docker Swarm is easier to set up and works well for smaller scale deployments.

13. **What are the benefits of using Docker Swarm for container orchestration?**
    Docker Swarm simplifies orchestration with an easy-to-configure system, native integration with Docker CLI, support for rolling updates, service discovery, and built-in load balancing. It is well suited for smaller clusters or environments that need to get started quickly.

14. **What are the key differences between Docker Swarm and Kubernetes?**
    Docker Swarm is typically easier to configure and less complex, making it ideal for smaller or simpler deployments.

    In contrast, Kubernetes offers a rich feature set and greater scalability for complex, enterprise-grade deployments but comes with a steeper learning curve and more involved management.

    i. **Complexity:** Swarm is simpler to set up and manage, good for smaller deployments. Kubernetes is more complex but offers more features.
    ii. **Features:** Kubernetes has a richer set of features for orchestration (more deployment strategies, auto-scaling options, self-healing, role-based access control) and a larger ecosystem.
    iii. **Community & Ecosystem:** Kubernetes has a much larger community, wider adoption, and a more extensive ecosystem of tools.

    Docker Swarm vs. Kubernetes:
    - **Choose Docker Swarm** – Ideal for simpler applications, smaller teams, and quick setup with familiar Docker commands.
    - **Choose Kubernetes** – Best for large-scale, mission-critical apps needing advanced features like auto-scaling, complex deployments, secrets management, and robust self-healing.

15. **Explain the key concepts in Docker Swarm (Manager, Worker, Service, Task).**
    Key Docker Swarm Concepts:
    - Manager Node – Handles orchestration, scheduling, scaling, and maintains cluster state.
    - Worker Node – Executes tasks assigned by managers and reports its status.
    - Service – Defines how containers run (image, replicas, ports, volumes, networks).
    - Task – The smallest unit, representing a single container and its commands.

16. **How do you scale a service in Docker Swarm?**
    You scale a service in Docker Swarm by updating the service definition to increase the number of desired replicas. You can do this using the docker service scale command, specifying the service name and the new number of replicas (e.g., docker service scale my-service=5). Docker Swarm managers will then create and schedule new tasks (containers) across the swarm nodes to reach the desired state.

17. **Docker Hub -** A **cloud-based registry** for storing and sharing Docker images. Developers use it to:
    - **Publish** public or private images.
    - **Pull** base images for container creation.
    - **Distribute** applications across development and production environments.

18. **What are the benefits of using Docker in a CI/CD pipeline:**
    - **Consistent Build Environment** – Ensures builds are uniform across different agents.
    - **Immutable Artifacts** – The built image is exactly what gets deployed.
    - **Simplified Deployment** – Tools only need to handle running Docker containers.
    - **Faster Builds** – Layer caching speeds up image creation.
    - **Isolation** – Prevents interference between different jobs.

19. **How would you use Docker in your local development workflow?**
    Using Docker in Local Development:
    - **Containerize Applications** – Run backend/frontend services in Docker containers.
    - **Containerize Dependencies** – Use containers for databases, queues, and other services.
    - **Use Docker Compose** – Manage multi-service setups easily with docker-compose.yml.
    - **Ensure Consistency** – Match local, staging, and production environments using the same images.

20. docker exec -it <container-id> /bin/bash **Command - Used to run commands inside a running container, lets you inspect, run commands, or troubleshoot inside the container.**

21. **How to inspect a running Docker container or an image:**
    a. **Inspect Container:** docker inspect <container-id> – Displays container's configuration, network, volumes, and state details.
    b. **Inspect Image:** docker inspect <image-id> – Provides information about image layers, configuration, and history.
    c. **View Logs:** docker logs <container-id> – Shows standard output and standard error streams of the container.

22. **Cleaning Up Unused Docker Resources:**
    - docker container prune – Removes all stopped containers.
    - docker image prune – Deletes dangling images; -a removes all unused images.
    - docker volume prune – Cleans up unused volumes.
    - docker network prune – Removes unused networks.
    - docker system prune – Cleans up containers, networks, images, and cache; -a removes all stopped containers and unused images.

23. **How do you backup or persist data in Docker Swarm deployments?**

Use volumes or bind mounts for persistence. **For stateful applications**, use external storage or volume plugins with backup, replication, and recovery features to ensure data durability across restarts and node failures.

24. **How do you update a service in Docker Swarm without downtime?**
    Use rolling updates to update one container at a time. By setting update parameters (e.g., delay, failure limits), Swarm ensures new instances are healthy before replacing old ones, maintaining service availability.

25. **How do you pass environment variables to containers?**
    Use the ENV instruction in the Dockerfile or pass them at runtime with the -e flag or via Docker Compose to configure container behavior dynamically.

26. **How does Docker Swarm handle rolling updates for services?**
    Docker Swarm performs rolling updates by incrementally replacing old tasks with new ones when a service is updated. You can configure the update policy, including delay and the number of concurrent updates, to ensure minimal downtime.

27. **How do you handle logging and monitoring in Docker environments?**
    Use Docker logging drivers to collect logs and forward them to systems like ELK/EFK. For monitoring, tools like Prometheus and Grafana or cloud solutions track container health and performance metrics.

28. **Describe how Docker manages resource allocation in a Swarm cluster.**
    Docker Swarm manages resource allocation by allowing you to set CPU and memory limits on services. The orchestrator schedules containers on nodes based on available resources, balancing workloads to optimize utilization and avoid resource conflicts.

29. **How is service discovery achieved in Docker Swarm?**
    Docker Swarm uses an internal DNS that assigns each service a DNS name, allowing services to find and communicate with each other without relying on fixed IP addresses.

30. **How does Docker Swarm handle load balancing for services?**
    Docker Swarm uses built-in ingress load balancing with a routing mesh, allowing any node to receive traffic on a published port and route it to a healthy task replica, regardless of its location.

31. **How can you secure Docker containers?**
    By running containers with least privilege, isolating them using namespaces, keeping images updated, scanning for vulnerabilities, monitoring activity, and applying network segmentation to limit communication.

KUBERNETES

What is Kubernetes and what problems does it solve?

- Is a container orchestration platform that automates the deployment, scaling, and management of containerized applications.
- addresses challenges like: sasrm
  - scheduling containers (across nodes),
  - auto-scaling based on demand,
  - self-healing from failures,
  - rollbacks, and
  - managing complex microservices in production.
- ~~Core architecture of Kubernetes (uses a master-worker architecture).~~
  - ~~The master node hosts the control plane: API server (cluster entry point), scheduler (assigns workloads), controller manager (maintains desired state), and etcd (stores cluster data).~~
  - ~~Worker nodes run Pods via kubelet (manages containers) and kube-proxy (handles networking).~~

~~**1. What is the concept of "Desired State" in Kubernetes?**~~

~~Kubernetes employs a declarative model where you define the desired state in YAML manifests (e.g., Pod replicas, container configurations, resource settings). The Control Plane, via controllers like Deployment or ReplicaSet, continuously monitors the cluster's actual state, compared it and reconciles any mismatches (e.g., crashed Pods, incorrect replica counts) by restarting Pods or scaling resources, ensuring self-healing, scalability, and consistency.~~

II. Components

1. What is a *Pod* in Kubernetes? What is its relationship with containers?

A **Pod** is the smallest unit in Kubernetes, encapsulating one or more containers that share network (localhost) and storage resources. It abstracts containers, allowing Kubernetes to orchestrate them as a single entity.

Containers within a Pod can communicate with each other via localhost and shared volumes. While Docker runs individual containers, Kubernetes orchestrates Pods.

2. What is a *Deployment* in Kubernetes? A *Deployment* (masur)

- manages a group of identical Pods, automating their creation, scaling (scaling up and down), updates (in a controlled way), and rollbacks (like previous version).
- ~~simplifies rollouts, rollbacks, and scaling operations, ensuring minimal disruption during version upgrades.~~

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 3
 selector:
  matchLabels:
   app: my-app
 template:
  metadata:
   labels:
    app: my-app
  spec:
   containers:
   - name: my-container
    image: my-image:1.0
    ports:
    - containerPort: 80
```

3. What is a *ReplicaSet* and how does it relate to Deployments? A *ReplicaSet*

- ensures a specified number of Pod replicas run continuously by automatically replacing pods if they fail or terminated this provides load balancing and fault tolerance.
- Deployments manage ReplicaSets. During rolling updates, a new ReplicaSet is created for the update; the Deployment then scales up the new version while scaling down the old one to maintain availability.

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-container
        image: my-image:1.0
        ports:
        - containerPort: 80
```

4. **What is a Kubernetes Service?** A ***Service***: (egu)
   - enable communication between different parts of your application or between applications and other services or users—without needing to track the changing IP addresses of Pods.
   - groups pods (containers) running the same application and exposes them as a single, stable endpoint (e.g., a DNS name or IP address) for other services or external clients to communicate with.
   - uses Kubernetes' built-in networking to route traffic to pods based on their labels.

Key Features:

- *Service Discovery*: Assigns a stable DNS name and IP address to a set of pods, ~~even as pods are created, destroyed, or rescheduled.~~
- *Load Balancing*: Distributes traffic across multiple pods to ensure scalability and reliability.
- *Pod Selection*: ~~Uses labels and selectors to identify which pods the service routes traffic to.~~
- ~~an abstraction that defines a logical set of Pods and a policy by which to access them, typically providing load balancing and service discovery.~~

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
```

~~**How It Works**: A Service uses Kubernetes' built-in networking to route traffic to pods based on their labels. It maintains a dynamic list of healthy pods and updates as pods change, ensuring high availability.~~

~~**Example Use Case**: A web application with multiple backend pods can use a ClusterIP Service to distribute HTTP requests evenly across those pods, while a LoadBalancer Service exposes a frontend to the internet.~~

Types of Services: CNLEH

- *ClusterIP (default):* Exposes the service within the cluster using an internal IP (e.g., for internal communication between microservices).
- *NodePort*: Exposes the service on a specific port of each node in the cluster, accessible externally via <NodeIP>:<NodePort>.
- *LoadBalancer*: Exposes the service externally using a cloud provider's load balancer (e.g., AWS ELB, GCP Load Balancer).
- *ExternalName*: Maps a service to an external DNS name without creating a proxy.
- *Headless*: Does not assign a ClusterIP, allowing direct communication with individual pods (useful for stateful applications like databases).

5. **What is a <u>StatefulSet</u>, and when should it be used over a Deployment?**
   - used for managing stateful applications.
   - Use for databases and message queues (like Mongo DB or Kafka), or other applications where instance identity and state persistence are important.
   - Used **Statefulset is for Stateful applications while Deployments are for stateless applications.**
   - Used **<u>StatefulSet</u>** instead of a **Deployment** when your application requires stable identities, persistent storage, and (ordered & graceful deployment and scalin)g.

6. **What is a DaemonSet, and what is its common use case?**
   - ensures that a copy of a pod runs on all (or selected) nodes in a cluster.
   - commonly used for logging daemons, node monitoring agents, or any service that should be present on every node.

III. Networking

1. **What is an Ingress in Kubernetes, and what is the role of an Ingress Controller?**

   Ingress in Kubernetes:
   - provides a way to expose multiple Services under a single external IP address, enabling efficient traffic management without needing multiple LoadBalancer Services.
   - defines rules for routing external HTTP/HTTPS traffic to Services, typically based on hostnames or URL paths.
   - Example: Route example.com/api to an API Service and example.com/web to a web Service.

   Ingress Controller:
   - specialized pod or set of pods (e.g., NGINX, Traefik, or AWS ALB Ingress Controller) that interprets Ingress rules and implements them by configuring a reverse proxy to direct traffic to the appropriate Services
   - manages external access to services in a Kubernetes cluster, typically via HTTP/HTTPS routing rules.
   - provides load balancing, SSL termination, and URL based routing, simplifying the management of external traffic.
   - Kubernetes does not provide a built-in Ingress Controller; you must deploy one to process the Ingress resource and manage routing

   How They Work Together: An Ingress resource specifies routing rules, and the Ingress Controller enforces them by dynamically updating its configuration to route external requests to the correct backend Services and their pods. This setup simplifies external access and centralizes traffic management.

   Example YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
```

2. **How is service discovery handled in Kubernetes?**

   Kubernetes provides built-in service discovery via Services, which assign stable IP addresses and DNS names to Pods. The cluster's DNS service maps Service names to their IPs, enabling reliable access despite Pod IP changes. Kube-proxy load-balances traffic to healthy Pods.

3. **How is Kubernetes networking configured among its various components?**

Kubernetes uses a flat networking model, allowing Pods to communicate without NAT. Container Network Interface (CNI) plugins like Calico, Flannel, or Weave Net manage connectivity and enforce network policies for security.

4. **Describe how Kubernetes implements load balancing between Pods?**
   Kubernetes handles load balancing between Pods primarily through Services and Ingress resources. A Service defines a set of Pods, selected by labels, and distributes traffic across them using a ClusterIP for internal access or a LoadBalancer for external access. Kube-proxy runs on each node and manages this distribution, typically using IPVS for efficient algorithms like round-robin. It ensures only healthy Pods, checked via readiness probes, receive traffic. For external HTTP traffic, an Ingress resource, paired with an Ingress Controller like NGINX, routes requests based on host or path and load balances across Pods. Features like session affinity and auto-scaling with the Horizontal Pod Autoscaler further enhance traffic management, ensuring scalability and reliability

IV. Configuration Management

1. **What are ConfigMaps and Secrets, and how are they used?**

   ConfigMaps:
   - Used to store non-confidential configuration data in key-value pairs or configuration files.
   - allow you to decouple configuration from the application code and Docker image.

   Secrets:
   - Similar to ConfigMaps but designed for storing sensitive information (passwords, API keys, tokens).
   - offer better security practices than storing secrets directly in images or ConfigMaps.

   ConfigMaps store non-sensitive configuration data separately from container images, while Secrets store sensitive data like passwords and tokens in an encoded form.

   Both are injected into Pods as environment variables or mounted files, allowing for dynamic configuration and enhanced security.

2. **How do you manage configuration for your microservices in Kubernetes?**
   Use Kubernetes ConfigMaps for non-sensitive configuration and Secrets for sensitive data.

   i. Define ConfigMaps and Secrets in YAML files.
   ii. Refer to these ConfigMaps and Secrets in your Deployment manifests to inject the data into your Pods as environment variables or mounted files. This decouples configuration from the Docker image.

V. Scaling

1. **How do you scale applications in Kubernetes?**

   Manual Scaling:
   - manually scale a Deployment by changing the replicas field in its manifest
   - using kubectl scale deployment <deployment-name> --replicas=<number>.

   Automatic Scaling using Horizontal Pod Autoscaler (HPA):
   - The HPA automatically scales the number of Pod replicas in a Deployment, ReplicaSet, or StatefulSet based on demands or observed metrics like CPU utilization, memory usage, or custom metrics. (if CPU usage goes above 70%, HPA will create more Pods; if demand drops, it scales down). You define minimum and maximum replica counts.
   - ensures that applications scale dynamically in response to demand.

VI. Storage

1. **How do you manage persistent storage for stateful applications in Kubernetes? How is storage handled in Kubernetes?**

By using Persistent Volumes (PVs), Persistent Volume Claims (PVCs), Storage Classes, and often StatefulSets to ensure data persistence across pod restarts, rescheduling, or failures.

Persistent storage is managed using PersistentVolumes (PV) and PersistentVolumeClaims (PVC).

- PVs are provisioned either statically or dynamically
- PVCs allow pods to request storage, ensuring that stateful applications persist data beyond the container's lifespan.

Persistent Volume (PV): (Cluster-wide storage)

- PV is a cluster-wide resource representing a piece of storage (e.g., NFS, cloud storage like AWS EBS, or local disks).
- It defines storage properties like capacity, access modes (ReadWriteOnce, ReadOnlyMany, ReadWriteMany), and reclaim policy.

Persistent Volume Claim (PVC): (Request for PV)

- A PVC is a request for storage by a pod or application, specifying size and access mode requirements.
- Kubernetes binds a PVC to a matching PV dynamically or statically.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi
```

Storage Class: (for dynamic provisioning) (Maintains persistent identity and storage for Pods)

- Defines storage provisioners (e.g., cloud provider-specific or custom) and parameters for dynamic PV provisioning.
- Allows automatic creation of PVs when a PVC requests storage from a specific Storage Class.
- Create a Storage Class to specify the provisioner and parameters (e.g., for AWS EBS, use gp2 or gp3 type).

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Retain
```

StatefulSet:

- A workload controller for stateful applications (e.g., databases like MySQL or MongoDB)
- Ensures pods have stable identities, predictable naming (e.g., pod-0, pod-1), and ordered deployment/scaling.
- Associates each pod with a unique PVC for persistent storage.
- Deploy a StatefulSet to manage stateful pods, associating each pod with a PVC using a volume claim template:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
```

```
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes:
        - ReadWriteOnce
      storageClassName: standard
      resources:
        requests:
          storage: 10Gi
```

Headless Service

- Used with StatefulSets to provide stable DNS names for individual pods (e.g., for peer discovery in databases).
- Define a Service with clusterIP: None to provide stable DNS entries for each pod:

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
```

Best Practices

- **Dynamic Provisioning**: Use Storage Classes for automatic PV creation to simplify management.
- **Reclaim Policy**: Set Retain for critical data to prevent accidental deletion or Delete for ephemeral storage.
- **Backup and Restore**: Implement regular backups (e.g., using Velero or cloud-native tools) to protect against data loss.
- **Access Modes**: Choose appropriate modes based on the application (e.g., ReadWriteOnce for single-pod access, ReadWriteMany for shared storage like NFS).
- **Monitoring**: Monitor storage usage and performance with tools like Prometheus and Grafana.
- **Cloud-Specific Storage**: Leverage cloud provider storage (e.g., AWS EBS, GCP Persistent Disk, Azure Disk) for managed solutions.

Example Use Case

For a MongoDB cluster:

- Deploy a StatefulSet with three replicas, each with its own PVC for data storage.
- Use a Headless Service for peer discovery (e.g., mongo-0.mongo.default.svc.cluster.local).
- Configure MongoDB to store data on the mounted PVC path and form a replica set using pod DNS names.

2. **What are Resource Requests and Limits in Kubernetes? Why are they important?**
   Requests define minimum resource needs for scheduling; limits cap usage to prevent overconsumption. They optimize resource allocation and cluster stability.
   - **Requests:** The minimum amount of CPU and memory a container needs. The Kubernetes scheduler uses requests to decide which node is suitable to place a Pod.
   - **Limits:** The maximum amount of CPU and memory a container can consume. If a container exceeds its memory limit, it is likely to be terminated. If a container exceeds its CPU limit, its CPU usage will be throttled.

VII. Security

1. **How do you secure communication between Kubernetes components?**
   Security is enforced using:
   - TLS (for encryption) for communication between cluster components
   - RBAC for authorization
   - network policies to restrict pod communication
   - Secrets for storing sensitive credentials

   Regular audits and security scans further enhance cluster integrity.

2. **What is Role-Based Access Control (RBAC) in Kubernetes and why is it used?**
   - mechanism to regulate access to Kubernetes resources based on the roles of individual users or service accounts
   - for securing the cluster and ensuring that users/applications only have the necessary privileges
   - to define fine-grained permissions (what actions a user/group can perform on which resources in which namespaces).

3. **Explain the importance of namespaces in Kubernetes.**
   - Namespaces are virtual clusters within a physical Kubernetes cluster that partition cluster resources into logical groups.
   - Namespaces are used for:
     - Resource Isolation – Separate workloads for teams or projects.
     - Access Control – Apply RBAC rules per namespace.
     - Environment Separation – Manage dev, test, and prod in one cluster.
     - Naming Conflict Avoidance – Same resource names in different namespaces.
     - Resource Quotas – Limit CPU, memory, and object use per namespace.
     - Better Organization & Scalability – Streamline large-scale management.

VIII. Reliability and Self-Healing

1. **How does Kubernetes handle self-healing of applications?**
   Kubernetes monitors Pod health with liveness/readiness probes, restarting unhealthy containers and ensuring only ready Pods receive traffic.
   ReplicaSets maintain the desired replica count, replacing failed Pods to recover from node or application failures.

   Kubernetes provides self-healing capabilities through its controllers and readiness/liveness probes

i. **Replication Controller/ReplicaSet:** ***(****recreate failed Pods)*
- Ensures the desired number of pod replicas are always running. If a pod fails or is deleted, the ReplicaSet automatically creates a new pod to replace it.
    - Example: A ReplicaSet with replicas: 3 will restart or reschedule pods to maintain three running instances.

ii. **Health Checks:**
- **Liveness Probes** *(restart unhealthy containers)*: Configured in the Pod specification, Detect if a pod is healthy (e.g., checking if an application is responsive). If a liveness probe fails, Kubernetes restarts the pod. Helps recover from deadlocks or unresponsive processes.
- **Readiness Probes** *(manage traffic routing to only healthy Pods):* Also configured in the Pod specification, these checks determine if a container is ready to serve traffic. A Pod is only considered ready if all its containers pass their readiness probes. Services only route traffic to ready Pods. Helps prevent traffic from being sent to containers that are still starting up or temporarily unavailable.

  Importance of Readiness Probes and Liveness Probes: They enable Kubernetes' self-healing and ensure that only healthy, ready Pods receive traffic, improving the reliability and availability of your application.

iii. **Pod Rescheduling:**
- If a node fails or becomes unreachable, Kubernetes' scheduler reschedules affected pods onto healthy nodes, provided there are sufficient cluster resources.
- The Kubelet on each node monitors pods and reports failures to the control plane.

2. **How do you backup and restore a Kubernetes cluster?**
- Backups include saving etcd data and critical configuration files (such as manifests and secrets).
- Tools like Velero facilitate backup and recovery of Kubernetes resources, enabling you to restore the cluster state in cases of failure or disaster.

IX. Deployment Strategies

1. **Explain the concept of Rolling Updates and Rollbacks in Kubernetes Deployments.**
- **Rolling Update:** The default update strategy for Deployments. When you update the Pod template, Kubernetes gradually replaces old Pods with new ones, ensuring the application remains available. You can configure parameters like maxUnavailable (maximum number of Pods that can be unavailable during the update) and maxSurge (maximum number of Pods that can be created above the desired number of replicas).
- **Rollback:** If a rolling update goes wrong (e.g., the new Pods fail health checks), you can roll back to a previous version of the Deployment using kubectl rollout undo deployment/<deployment-name>.

2. **What strategies exist for deploying updates to applications in Kubernetes?**
Options include 1. rolling updates (incremental, update pods incrementally), 2. blue green (full switch), 3. canary (gradual rollout). These strategies help minimize downtime and reduce risk during updates.

X. Monitoring and Troubleshooting

1. **How do you monitor applications running in Kubernetes/ monitor Kubernetes cluster?**
Monitoring is achieved through the use of:
   i. **Prometheus** (for metrics) : Open-source system scrapes metrics from nodes, pods, and services.
   ii. **Grafana** (for visualization): Visualizes Prometheus metrics on dashboards.
   iii. **Health Checks**: Liveness and Readiness probes monitor application health.
   iv. **Metrics Server**: Collects CPU/memory metrics from kubelet for HPA.
   v. **Logging Agents**: Fluentd/Logstash collect container logs for centralized systems (Elasticsearch/Splunk).

2. **How do you troubleshoot issues in a Kubernetes cluster?**
to troubleshoot/diagnose and resolve issues.

- Leverage commands like kubectl get events, kubectl describe, kubectl logs, and kubectl exec (Execute commands inside a running container for debugging),
- monitoring tools such as Prometheus, Grafana, to collect metrics and set up alerts.,
- Distributed Tracing such as Zipkin to trace requests across microservices running in the cluster

XI. Best Practices

1. **What best practices do you follow when deploying applications on Kubernetes.**
   - Use **declarative configs** (YAML manifests)
   - Set **resource /limits**
   - Applying **network policies** for traffic control
   - Manage **Secrets securely**
   - Leverage probes, autoscaling.
   - Employing robust logging, monitoring,
   - Use **Canary or Blue-Green deployments** to reduce risk.

   These practices ensure that applications are secure, scalable, and resilient to failures.

XII. Tools and Features

1. **What is the significance of the *kubectl* command-line tool?**
   - CLI tool to interact with Kubernetes API /Kubernetes clusters.
   - allows developers: to deploy applications, inspect cluster resources, manage configurations, and troubleshoot issues.
2. **Explain the role of etcd in a Kubernetes cluster.**

   etcd is a distributed key value store used by Kubernetes to persist cluster state data, configuration details, and metadata.
3. **What is Helm, and how does it simplify application deployment?**

   Helm is a
   - package manager for Kubernetes that allows applications to be defined, installed, and managed using chart packages.
   - Helm charts encapsulate Kubernetes manifests, making it easy to version, share, and deploy complex applications with a single command.
4. **How would you deploy a Spring Boot microservice (as a Docker image) to Kubernetes?**

   a) Create a Docker image for the Spring Boot application.

   b) Push the Docker image to a registry.

   c) Write Kubernetes YAML manifests: (Deployment, Service, Ingress)
      - Deployment: Define the desired number of replicas, the Docker image to use, port, resource requests/limits, liveness/readiness probes.
      - Service: Define a Service (e.g., ClusterIP) to expose the Pods internally for inter-service communication. Expose the app externally via LoadBalancer.
      - Optional: Ingress: Define an Ingress resource to expose the service externally via HTTP/S, if needed.

   d) Deploy using command kubectl apply -f <file>.yaml

   e) Verify with kubectl get pods and kubectl get svc.